

Pipeline Design Specification

Retail Customer Intelligence Platform — definition of every data pipeline from raw CSV → Bronze → Silver → Gold (star schema + RFM) → ML training & batch scoring, including **load strategy (full vs incremental)**, grain, partitioning, idempotency, dependencies, and schedule for each entity.

AIO Conquer 2026 · Module 01 — companion to `source_to_target_mapping.md`. Where current code and the recommended target differ, both are stated explicitly.

0. Design Principles

Principle	Rule
Idempotency	Re-running a pipeline for the same partition produces the same result; never double-counts.
Partitioning	Bronze/Silver are partitioned by shifted <code>year_month=YYYY-MM</code> (Hive style).
Immutability (Bronze)	Bronze is the system of record; once written, a month partition is never edited in place.
Watermark	The partition key <i>is</i> the watermark — "what months already exist" decides what to load.
Late-arriving data	Handled by re-running (overwriting) the affected month partition, not by appending.

Full vs Incremental — decision matrix

Pattern	Use when	Entities here
Incremental (append partition)	Event/transaction grain, naturally time-partitioned, immutable history	<code>bronze_transactions</code> , <code>silver_transactions</code> , <i>(target)</i> <code>fact_transactions</code>
Full refresh (rebuild table)	Small reference data, or aggregates whose value depends on <i>all</i> history	<code>dim_*</code> , <i>(current)</i> <code>fact_transactions</code>
Full recompute (snapshot)	Metric is relative to a moving reference <code>(MAX(date))</code> , so every row can change each run	<code>mart_rfm</code> , <code>mart_churn_scores</code>
Full retrain	Model must learn from the whole labeled history on a cadence	churn model (<code>ml/train.py</code>)

1. Pipeline Inventory

#	Pipeline	Entity / output	Load type	Trigger / schedule	Idempotent by
P1	<code>bronze_ingest</code>	<code>bronze_transactions</code>	Incremental (per month, skip existing)	<code>@daily</code> / on new raw drop	partition exists check
P2	<code>silver_transform</code>	<code>silver_transactions</code>	Incremental (per month, skip existing)	after P1	partition exists check
P3	<code>dbt run</code> (stg+int)	<code>stg_*</code> , <code>int_*</code> (views)	Recomputed (views, no	after P2	views are stateless

#	Pipeline	Entity / output	Load type	Trigger / schedule	Idempotent by
			storage)		
P4	dbt run (dims)	dim_customer/product/country/date	Full refresh (table)	after P3	full rebuild
P5	dbt run (fact)	fact_transactions	Full refresh (current) → Incremental (target)	after P4	full rebuild / partition
P6	dbt run (rfm)	mart_rfm	Full recompute (snapshot)	after P5	full rebuild
P7	dbt test	data-quality gate	— (validation only)	after P3–P6	n/a
P8	ml train	churn model + MLflow run	Full retrain	@weekly (recommended)	new MLflow run + registry
P9	ml score	mart_churn_scores	Full recompute (snapshot)	@daily after P6	overwrite parquet

2. Per-Entity Specification

P1 · bronze_transactions — INCREMENTAL

- **File:** src/etl/bronze_ingest.py · **Grain:** transaction line · **Storage:** data/bronze/year_month=YYYY-MM/data.parquet
- **Load logic:** ingest_all() discovers all shifted months and iterates **newest-first**; already_ingested(ym) skips a month whose partition already exists.
- **Why incremental:** transactions are append-only events; once a month is landed it does not change.
- **Backfill / reprocess:** delete the target year_month= folder and re-run --month YYYY-MM (overwrite-by-replace).
- **Idempotency:** running --all twice is a no-op for existing months.
- **Audit:** ingested_at per row + _ingestion_log.csv.

P2 · silver_transactions — INCREMENTAL

- **File:** src/etl/silver_transform.py · **Grain:** transaction line · **Storage:** data/silver/year_month=YYYY-MM/data_silver.parquet
- **Load logic:** iterates Bronze partitions newest-first; skips a month whose Silver partition exists.
- **Transforms:** dedup, line_amount, noise filter (price<=0 & not cancellation), calendar columns (see STM §3).
- **Quality:** per-partition quality_report.json + _quality_log.jsonl (TabularDataQuality).
- **Reprocess:** delete the Silver year_month= folder; re-run for that month.

P3 · stg_silver__transactions, int_transactions__prepared — VIEWS (recomputed)

- **Files:** dbt/models/staging/, dbt/models/intermediate/ · **Materialization:** view.
- No physical storage; resolved at query time from the full Silver parquet glob. Always reflects current Silver.

P4 · dim_customer / dim_product / dim_country / dim_date — FULL REFRESH

- **Files:** `dbt/models/marts/dim_*.sql` · **Materialization:** `table` (rebuilt every `dbt run`).
- **Why full:** dimensions are small; surrogate keys assigned by `ROW_NUMBER()`.
- **SCD:** effectively **Type 1** (no history) — a changed `description/segment` overwrites.
- ⚠ **Caveat:** because surrogate keys come from `ROW_NUMBER()`, they are **not stable** across rebuilds. Acceptable while fact is also full-rebuilt (P5 current), but must be addressed if fact becomes incremental (see P5 target).

P5 · `fact_transactions` — FULL REFRESH (current) → INCREMENTAL (target)

- **File:** `dbt/models/marts/fact_transactions.sql` · **Grain:** transaction line (purchases **and** cancellations).
- **Current:** `table` materialization, full rebuild each run; FKs via `LEFT JOIN` to dims; `transaction_sk = ROW_NUMBER()`.
- **Target (incremental):**
 - Switch to `materialized='incremental'` partitioned on `date_sk/year_month`, only building the latest open month(s).
 - Replace `ROW_NUMBER()` degenerate key with a **deterministic** surrogate (hash of `invoice||stock_code||tx_date`) so re-runs don't reshuffle keys.
 - Requires **stable dimension keys** first (natural-key lookup or persisted SK map) — list as a prerequisite ticket.
- **Net revenue rule preserved:** `SUM(line_amount)` over the full fact (cancellations are negative).

P6 · `mart_rfm` — FULL RECOMPUTE (snapshot)

- **File:** `dbt/models/marts/mart_rfm.sql` · **Grain:** customer.
- **Why full recompute:** `recency_days` is measured against `ref_date = MAX(invoice_date)`; this moving reference means every customer's R/F/M and segment can change each run → cannot be incremental.
- Built from `is_valid_purchase` rows only; quintile `NTILE(5)`; 11 segment labels.

P8 · churn model — FULL RETRAIN

- **File:** `ml/train.py / ml/pipeline.py --mode train` · **Source:** Silver layer (not the dbt marts).
- **Feature window:** features computed on the observation period; label = no purchase in last `EVALUATION_WINDOW_DAYS = 90` days (`ml/config.py`, `docs/ml_document/ml_design.md`).
- **Strategy:** full retrain on a cadence (recommended `@weekly`) → logs params/metrics to **MLflow** (`churn-prediction` experiment) → QA gate `AUC ≥ 0.80` → save `ml/artifacts/model.joblib` + register.
- **Why full:** small dataset (~5k customers); retraining the whole history is cheap and avoids drift.

P9 · `mart_churn_scores` — FULL RECOMPUTE (snapshot)

- **File:** `ml/score.py / ml/pipeline.py --mode score` · **Grain:** customer · **Storage:** `data/gold/mart_churn_scores/mart_churn_scores.parquet`.
- Loads latest model, scores **all** customers (`churn_probability`, `churn_flag` at F1-optimal threshold, `risk_tier`), overwrites the parquet. Recommended cadence `@daily` after P6.

3. ML Data Extraction → Training Flow

```

Silver (clean transactions)
  | ml/features.py  build_feature_matrix(mode="train")
  ▼
Feature matrix (~5k customers × 31 features) + churn label (90-day window)
  | stratified 80/20 split
  ▼
ml/train.py → XGBoost + CV → MLflow log (params, cv_auc)
  |
  ▼
ml/evaluate.py → test AUC, threshold opt → QA gate (AUC ≥ 0.80)
  | pass → save artifacts + register

```

```
▼
ml/score.py  build_feature_matrix(mode="score")  → batch score ALL customers
▼
data/gold/mart_churn_scores  → Power BI / retention export
```

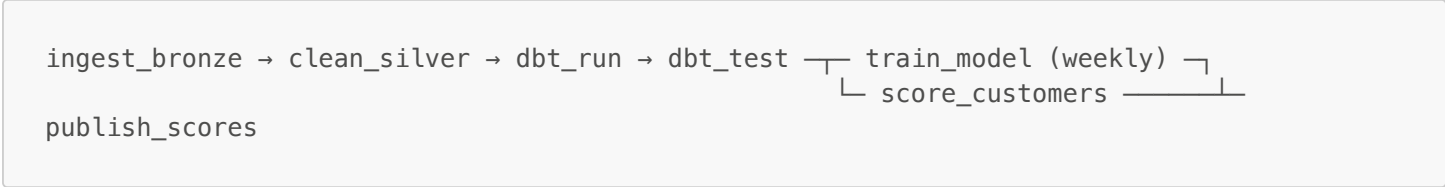
- **Train vs Score feature parity:** same `FEATURE_COLUMNS`; `mode="train"` adds the label, `mode="score"` omits it. This guarantees no train/serve schema skew.
- **Reproducibility:** `RANDOM_STATE = 42`, fixed `TEST_SIZE = 0.20`.

4. Target Orchestration DAG (Airflow)

Current (`airflow/dags/retail_pipeline_dag.py`):

```
ingest_bronze → clean_silver → dbt_run → dbt_test
```

Target (extend with the ML branch — Sprint 3 gap):



Task	Command	Cadence
ingest_bronze	python -m src.etl.bronze_ingest --all	daily
clean_silver	python -m src.etl.silver_transform --all	daily
dbt_run	dbt run	daily
dbt_test	dbt test	daily (gate)
train_model	python -m ml.pipeline --mode train	weekly
score_customers	python -m ml.pipeline --mode score	daily
publish_scores	export mart_churn_scores → serving	daily

`dbt_test` is a **quality gate**: downstream ML tasks should not run if it fails (current DAG runs them unconditionally — wire `trigger_rule` once ML tasks are added).

5. Backfill & Reprocessing Runbook

Scenario	Action
New month of raw data	<code>bronze_ingest --all</code> → <code>silver_transform --all</code> (skips existing, lands new) → <code>dbt run</code> → <code>dbt test</code> .
Fix a single month	Delete <code>data/bronze/year_month=YYYY-MM/</code> and <code>data/silver/year_month=YYYY-MM/</code> , re-run <code>--month YYYY-MM</code> , then <code>dbt run</code> .
Full rebuild	Delete <code>data/bronze/</code> , <code>data/silver/</code> , <code>data/retail.duckdb</code> ; re-run from P1.
Model refresh only	<code>ml.pipeline --mode train</code> then <code>--mode score</code> (no data reload needed).

Scenario	Action
Re-score only	<code>ml.pipeline --mode score</code> (reuses saved model).

6. Freshness & SLA (recommended)

Entity	Freshness target	Monitored by
<code>bronze_transactions</code>	latest month present within 24h of raw drop	<code>_ingestion_log.csv</code> row count/run
<code>silver_transactions</code>	same run as Bronze	<code>_quality_log.jsonl</code>
Gold marts	rebuilt every daily run	<code>dbt test</code> pass = 65 tests
churn model	AUC $\geq$ 0.80 each retrain	MLflow <code>qa_gate_passed</code> tag
<code>mart_churn_scores</code>	refreshed daily	row count = customer count